

DATA 266 — Homework 3

DATA 266 — Homework 3

SID4 = 1605 | SEED = 1605

Notebook: `attention_and_prompting.ipynb` (single notebook, both parts). Two trained models for Part B (unmasked attention and causal-masked attention) — no separate hyperparameter-sweep model was needed for this assignment. Full transcripts and tables: `METRICS.md` and `RUN_LOG.txt`.

1. Prompt Engineering (5 marks)

All twelve prompts were called from code with **LangChain** (`ChatPromptTemplate` + `ChatOllama`), not a chat-UI screenshot. Backend: a **local llama3.1:8b** model served by Ollama — this machine has no Anthropic/OpenAI/Google API key configured, so calling a local model in code was the only way to satisfy “use code to call these prompts” without handing out a key. Two examples per technique, drawn from a math word problem (correct answer: **28**) and a logical-reasoning problem (correct answer: **6**), plus one text-classification example for Meta-Prompting and two puzzle problems for Tree of Thoughts.

What differed between techniques

Zero-Shot answered the logic question correctly (“Yes”) but got the arithmetic wrong (**32**, with no reasoning shown to audit). **Few-Shot** made a different kind of mistake: primed with three worked exemplars and then asked the real question, it answered **21** — which is not a number derivable from the actual problem at all, but is the *first exemplar’s own answer*. The model pattern-matched the exemplar block instead of transferring its method to the new question — a genuine “exemplar leakage” failure, not just an occasionally-wrong number.

Chain-of-Thought (few-shot, with a worked exemplar) got both examples right, including correctly transferring the “all but N” idiom from a sheep exemplar to a completely new books question. **Zero-Shot CoT** solved the math problem identically well, but on the logic question it reasoned correctly for two steps (“all but 6 means 6 remain”) and then **contradicted itself**, going on to compute $30 - 6 = 24$ and reporting 24 as final — a self-contradiction that never appeared in the few-shot version, where the worked exemplar anchored the model to stop at the right step.

Meta-Prompting — a single reusable system instruction (“identify problem type, plan, execute, then state a Final Answer”) — was the only technique applied verbatim across two *different* task types (the math problem and a nuanced movie-review sentiment classification) and got both right, correctly weighing the review’s positive/negative tension before committing to “Mixed.”

Tree of Thoughts (3 sampled branches + an evaluation/selection call) failed on both examples, and failed instructively. For the 24-game puzzle (numbers 3, 5, 7, 11), a valid solution exists — $(7 - 3) * (11 - 5) = 4 * 6 = 24$ — but none of the 3 branches found it; they cycled through repeated wrong arithmetic until hitting the output-length cap, and the evaluation step also failed to discover the correct expression. For a logic-grid puzzle (three friends / three pets / three cities), all 3 branches came back **nearly word-for-word identical** despite temperature 0.8 — no real solution diversity — and the shared answer got the pet right

(cat) but the city wrong (said “Boston,” which is actually the dog-owner’s city per the puzzle’s own clue; the correct city is Chicago). Five LLM calls bought nothing over one here.

Overall: CoT and Meta-Prompting were the only techniques that were reliably correct on problems with a non-obvious decomposition, matching the standard intuition. But the results also undercut a naive “more structure always helps” story: Zero-Shot-CoT still self-contradicted, Few-Shot still latched onto the wrong exemplar, and Tree-of-Thoughts — with no real search or verification behind it — sampled the same wrong reasoning three times instead of genuinely exploring alternatives. Full transcripts are in `METRICS.md` and the executed notebook.

2. Self-Attention and Causal Masking (10 marks)

Implemented single-head scaled dot-product self-attention from scratch in PyTorch (Section 3.2, *Attention Is All You Need*) — only `nn.Embedding`, `nn.Linear`, matrix multiplication, softmax, and cross-entropy loss; no `nn.MultiheadAttention` / `nn.Transformer` / HuggingFace classes.

Setup: the assignment’s fixed 5-sentence paragraph, tokenized at the word level (**51** tokens, vocabulary size **41**). Trainable token + positional embeddings (`d_model = 32`) feed a single self-attention layer ($Q = XW_Q$, $K = XW_K$, $V = XW_V$, scores = $QK^T / \sqrt{d_k}$), softmax, weighted sum of V), followed by a linear head back to vocabulary logits. Two model instances were trained independently and identically (Adam, $lr=3e-3$, 300 epochs, autoregressive next-token cross-entropy) — one with no mask (Part 1) and one with a causal mask applied to the scores *before* softmax (Part 2), so the “two trained models” this assignment calls for are the unmasked and masked variants, not a hyperparameter sweep.

Both losses started near $\ln(41) = 3.71$ (random-guess loss for a 41-token vocabulary) and dropped to **0.0027** (unmasked) and **0.0032** (causal) by epoch 300 — confirming the embeddings and W_Q , W_K , W_V projections were genuinely learned, not left at random initialization, so the heatmaps below reflect real learned structure.

Unmasked self-attention (after training)

Attention is spread across full rows, including columns to the right of the diagonal — every token is free to attend to tokens that come later in the sequence, as expected with no mask.

Causal (masked) self-attention (after training)

The entire strict upper triangle is exactly 0 — verified numerically inside the notebook via `np.triu(attn, k=1).sum()`, which prints **0.0000000000**, not just eyeballed on the plot. Row 0 (the first token) can only attend to itself, producing a single weight-1.0 spike at column 0; each subsequent row gains exactly one more available column, giving the characteristic triangular pattern of decoder self-attention. This directly demonstrates the mechanism Section 3.2.3 of *Attention Is All You Need* describes: causal masking prevents a position from using information about future tokens it would not have access to at generation time.

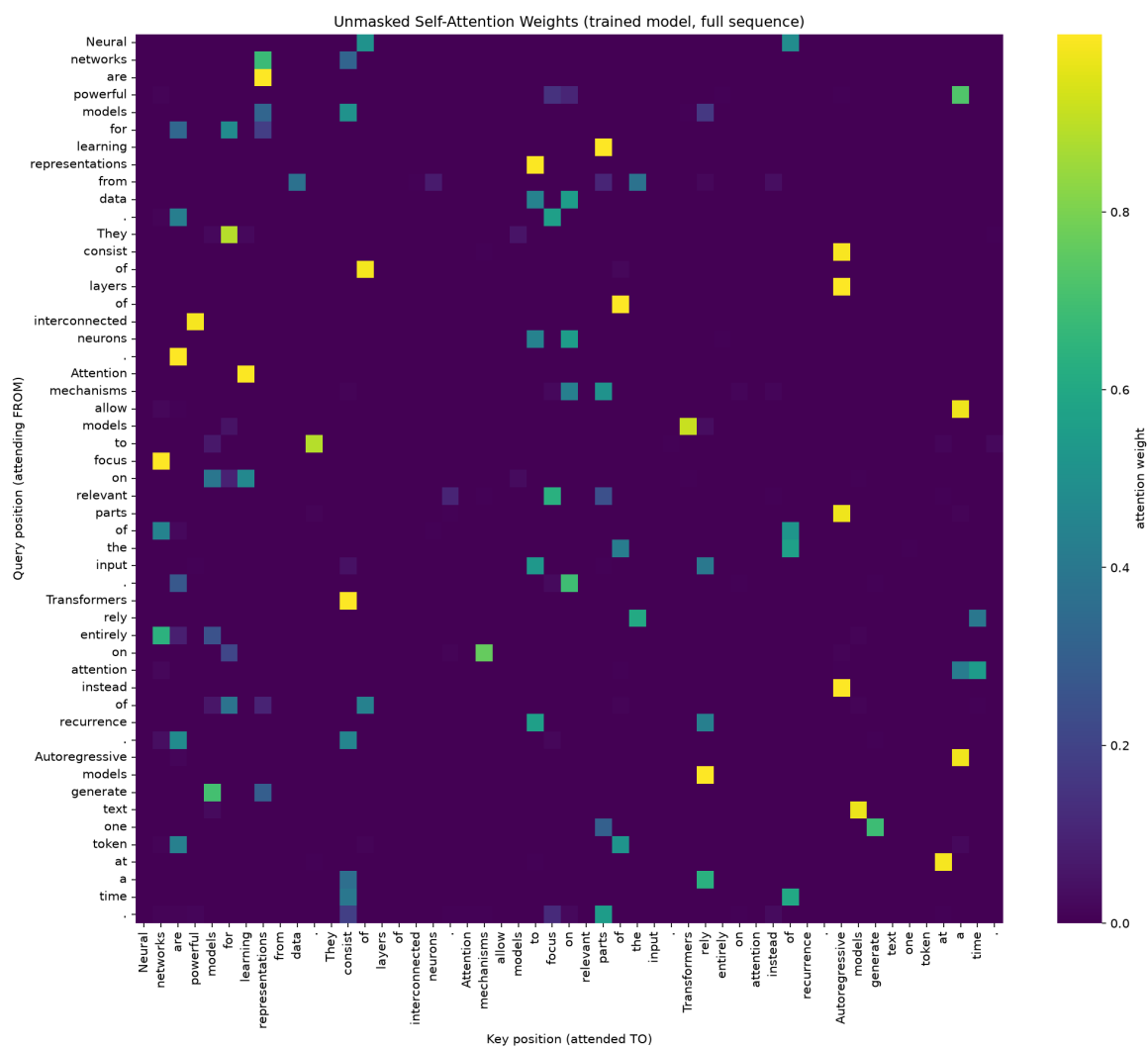


Figure 1: Unmasked self-attention heatmap

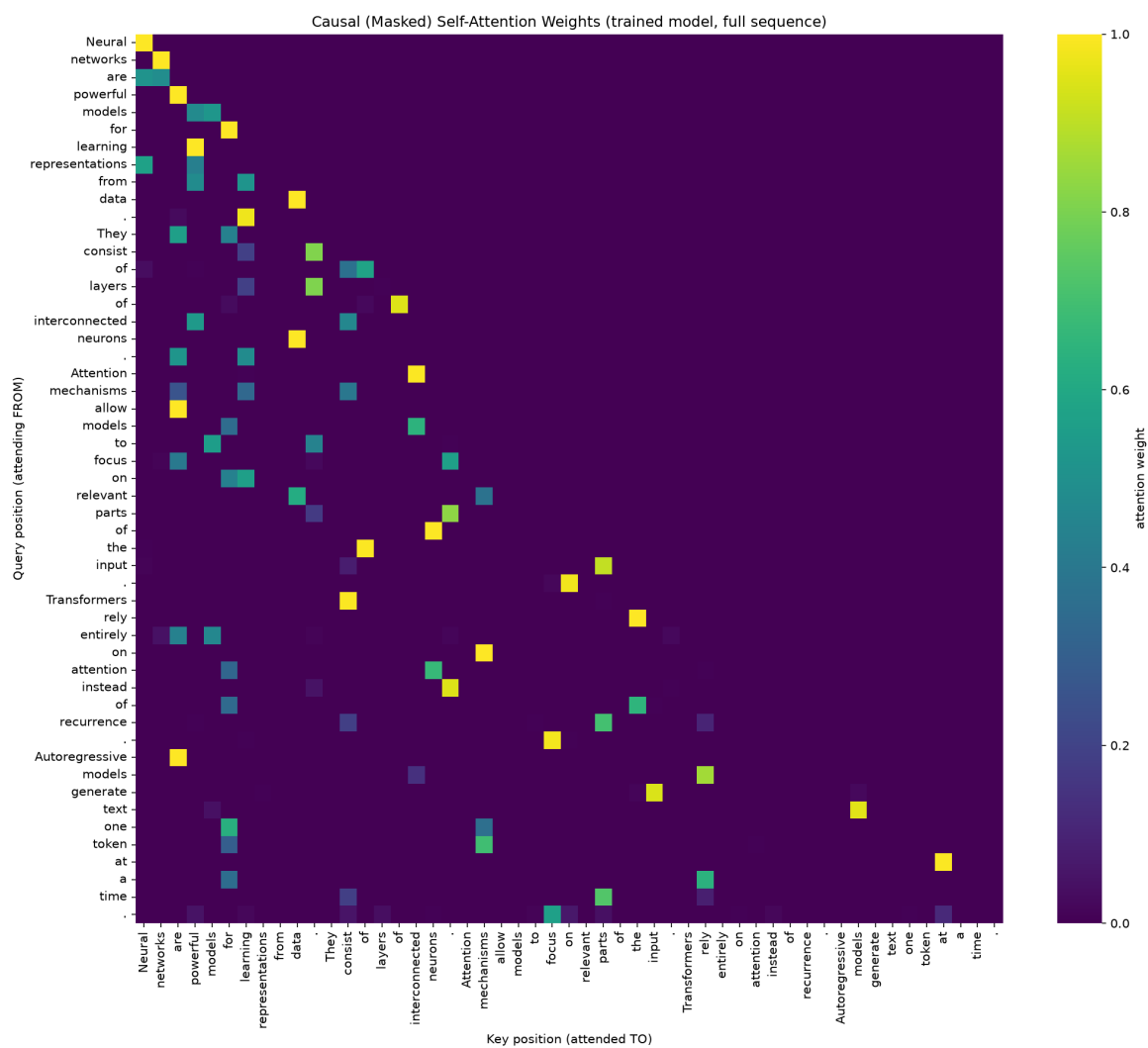


Figure 2: Causal self-attention heatmap